

SD3589CV100 NXP PMIC

开发参考

文档版本　1.0.0

发布日期　2025-04-07

概述

本文档主要介绍 SD3589CV100芯片上NXP PMIC开发参考。

版本说明

与本文档相对应的产品版本如下。

产品名称	产品版本
SD3589C	V100
SD3589D	V100

用户

本文档主要适用于以下工程师：

- ✧ 技术支持工程师
- ✧ 软件工程师

修订记录

修订记录列出了各文档版本间发生的主要更改。最新版本的文档包含以前所有文档版本的更新内容。

版本	修订日期	修订说明
v1.0.0	2025.04.07	首个NXP PMIC开发参考文档

 版权所有 © 上海云集互联有限公司

目 录

目录

开发参考 1
目 录 III
插图目录 IV
表格目录 VIII
1. 概述 1
1.1. 概述 1
1.1.1. FS85概述 1
1.1.2. PF82概述 1
1.2. 相关术语 1
1.3. 功能描述 2
1.3.1. FS85功能描述 2
1.3.2. PF82功能描述 3
1.4. 电源域描述 5
2. 配置说明 7
2.1. FS85配置 7
2.2. PF82配置 7
2.2.1. PF82 linux驱动介绍 8
2.2.2. PF82 linux总体框架 11
2.2.3. PF82 linux驱动配置 12
2.3. 编译配置 17
2.4. 烧录配置 18
3. 调试说明 19
3.1. FS85调试说明 19
3.2. PF82调试说明 19
3.2.1. OTFL PF82调试说明 19
3.2.2. Linux PF82调试说明 19

版权所有 © 上海云集互联有限公司

插图目录

图 1-1 FS85功能框图.....2
图 1-2 PF82功能框图.....4
图 1-3 PMIC电源树.....6
图 2-1 MFD框架.....8
图 2-2 regmap框架.....9
图 2-3 regulator框架.....10
图 2-4 linux pf8200驱动架构.....12

版权所有 © 上海云集互联有限公司

表格目录

表格 1-1 相关术语..... 1

版权所有 © 上海云集互联有限公司

1. 概述

1.1. 概述

本文档主要介绍了SD3589CV100 DEMO2开发板NXP PMIC 配置和调试方法，为用户提供PMIC模块开发指引。

1.1.1. FS85概述

FS85家族作为NXP最新一代的一级电源管理芯片系列，其电源输出能力更强， FS85符合功能安全ASIL-D标准。FS85系列是一款汽车功能安全的多输出电源集成电路，专注于雷达、视觉、ADAS域控制器、无线电和信息娱乐应用。它包括多个开关模式和线性稳压器。提供外部频率同步输入输出，优化系统电磁兼容性能。

FS85的最大输入电压是 60V，可应用于新能源汽车的 12V 或者 24V 电源系统中。FS85 包括 1 路高压 Buck，3 路低压 Buck，1 路低压 Boost，2 路低压 LDO，可通过 I2C 或者 SPI 接口与主机通讯。主控 MCU 在运行过程中需要定时给 FS85 发送喂狗信息，通过这种方式来监测主控 MCU 是否宕机。一旦超时没有喂狗，FS85 就会认为主控 MCU 出现问题，就会立即采取一系列措施，比如产生中断信号、拉低复位管脚、断电等。

1.1.2. PF82概述

PF82家族是NXP新一代搭配高性能处理器二级电源产品，其特点在于多路高性能的电源输出，且通过OTP配置，可灵活定制其默认电压及上下电时序。PF8200满足功能安全ASIL-B的系统要求，可为系统提供多达12路符合ASIL-B的电源输出。大大减轻了功能安全的硬件设计负担。广泛应用于智能座舱、虚拟仪表、ADAS及域控制器等系统之中。凭借PF8200出色的电源性能，I.MX8系列、S32V2系列、Layerscape等NXP的高性能处理器应用参考设计中，都应用PF82为其系统供电；同时，对于FPGA以及众多竞争对手的处理器平台，在自动驾驶、域控制、车载娱乐等领域，PF8200也有很多量产案例。

PMIC PF82设有七个高效的降压转换器（BUCK1-BUCK7）和四个线性稳压器（LDO1-LDO4），可为处理器、存储器和其他外设供电。内置的一次性可编程（OTP）存储器可存储关键的启动配置，大大减少了通常用于设置输出电压和外部稳压器序列的外部元件数量。启动后，稳压器参数可通过高速I2C进行调节，从而为不同的系统状态提供较高的灵活性。

1.2. 相关术语

表格 1- 1 相关术语

术语	介绍
PMIC	PMIC(Power Management Integrated Circuits)电源管理集成IC，负责系

版权所有 © 上海云集互联有限公司

	统各模块供电管理。
LDO	LDO(Low Dropout Regulator)，低压差稳压器，又称低压差线性稳压器、低压降稳压器，是线性直流稳压器的一种，用途同是提供稳定的直流电压电源。相比于一般线性直流稳压器，低压差稳压器能于更小输出输入电压差的情况下工作。
BUCK	BUCK是DC-DC中的一种，DC-DC的意思是直流变（到）直流（不同直流电源值的转换），只要符合这个定义都可以叫DC-DC转换器，包括LDO。DC-DC主要有BUCK（降压），BOOST（升压），BUCK-BOOST（升降压）三种。
Regulator	linux 内核对 LDO、 DC-DC 的控制核心。

1.3. 功能描述

1.3.1. FS85功能描述

FS85 功能框图如下图所示：

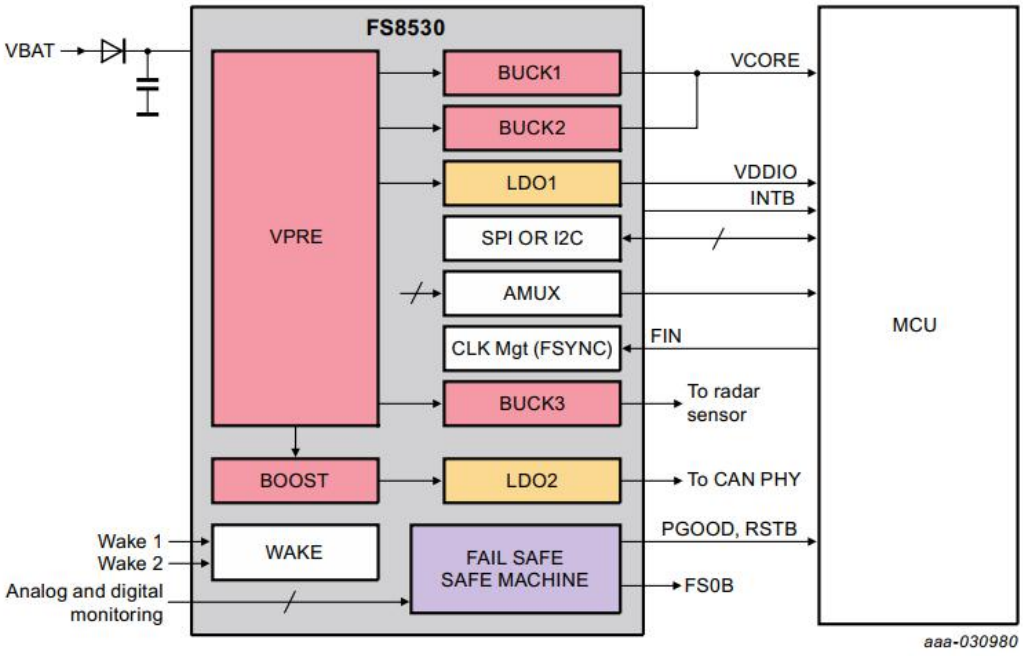


图 1- 1 FS85功能框图

FS85 具有以下功能特点：

- 适用于12V和24V应用的最高60V直流输入电压
- 带外部MOSFET的VPRE同步降压控制器可配置的输出电压、开关频率以及高达10A峰值的电流能力。

版权所有 © 上海云集互联有限公司

- 低电压集成同步BUCK1转换器，专用于微控制器核心电源，具有静态电压调节能力。可配置的输出电压以及高达3.6A峰值的电流能力
- 低电压集成同步BUCK2转换器。可配置的输出电压以及高达3.6A峰值的电流能力。与BUCK1多相配合，可将单轨的电流能力扩展至高达7.2A峰值。具有静态电压调节能力
- 低电压集成同步BUCK3转换器。可配置的输出电压以及高达3.6A峰值的电流能力
- 带集成低端开关的BOOST转换器。可配置的输出电压以及高达1.5A峰值的最大输入电流
- 电磁兼容性优化技术，包括开关电源频率同步、展频、上升沿/下降沿速率控制、手动频率调节
- 2个线性电压调节器，用于微控制器I/O和ADC电源以及外部物理层。可配置的输出电压以及高达400mA直流的电流能力
- 关闭模式（断电）下具有非常低的静态电流（典型值为10μA）
- 2个输入引脚，用于唤醒检测和电池电压感应
- 通过32位SPI或I2C接口以及CRC进行设备控制
- 电源同步引脚，用于操作2个FS85设备或FS85加一个外部PMIC
- 从ASIL B到ASIL D的可扩展产品组合，具有独立监测电路、专用于微控制器监测的接口、简单和挑战者看门狗功能、电源良好、复位和中断、内置自检、故障安全输出
- 通过一次性可编程（OTP）进行配置。在工程模式下支持项目开发过程中的自定义设置，以实现原型开发

1.3.2. PF82功能描述

PF82 功能框图如下图所示：

版权所有 © 上海云集互联有限公司

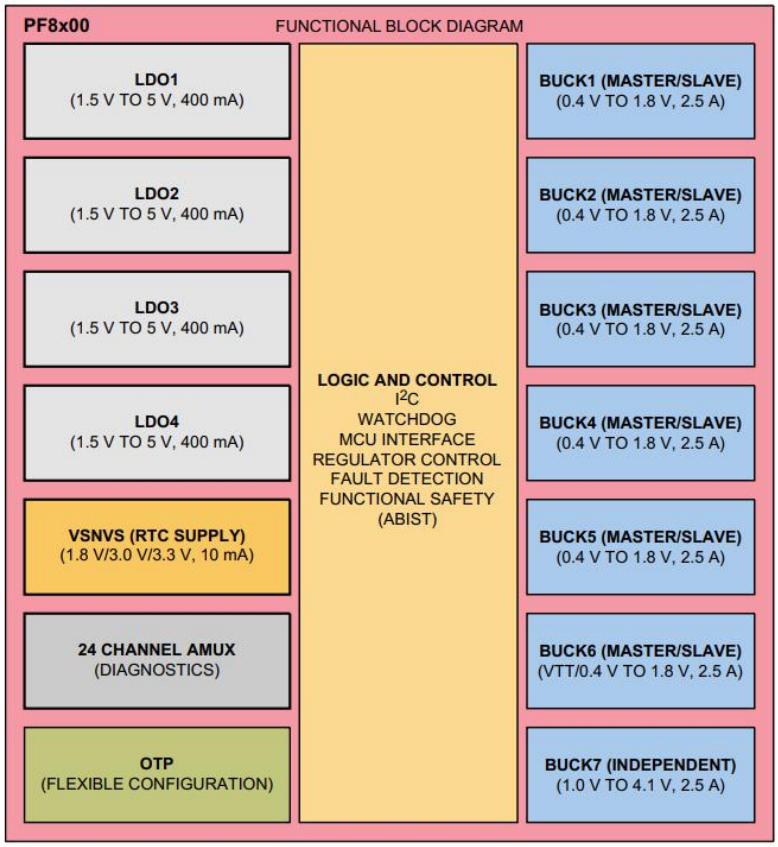


图 1- 2 PF82功能框图

PF82 具有以下功能特点：

- 降压稳压器（Buck）
  - SW1、SW2、SW3、SW4、SW5、SW6：输出电压范围为0.4V至1.8V，最大输出电流为2500mA，精度高达1.5%。
  - SW7：输出电压范围为1.0V至4.1V，最大输出电流为2500mA，精度为2%。
  - SW1、SW2、SW3、SW4、SW5和SW6支持动态电压调节
  - SW1和SW2可配置为双相稳压器
  - SW3和SW4可配置为双相稳压器
  - SW5和SW6可配置为双相稳压器
  - SW1、SW2和SW3可配置为三相稳压器，电流能力高达7.5A
  - SW1、SW2、SW3和SW4可配置为四相稳压器，电流能力高达10A
  - SW6支持VTT终端模式
  - 可编程电流限制，持扩频和手动调节开关频率
- 低压差线性稳压器（LDO）
  - LDO1：输出电压范围为1.5V至5.0V，最大输出电流为400mA，精度为3%，可选负载开关

版权所有 © 上海云集互联有限公司

模式

- LDO2：输出电压范围为1.5V至5.0V，最大输出电流为400mA，精度为3%，可选负载开关模式，支持硬件/软件控制选择
- LDO3：输出电压范围为1.5V至5.0V，最大输出电流为400mA，精度为3%，可选负载开关模式
- LDO4：输出电压范围为1.5V至5.0V，最大输出电流为400mA，精度为3%，可选负载开关模式

- 实时时钟（RTC）LDO/开关电源
  - RTC电源：VSNVS支持1.8V/3.0V/3.3V，最大电流为10mA
  - 带电池备份的存储器：包括硬币电池充电器，可编程充电电流和电压
- 系统特性
  - 快速PMIC启动
  - 高级状态机：实现与处理器的无缝接口
  - 支持高速I2C接口（高达3.4MHz）
  - PGOOD监控
  - 用户可编程的待机和关闭模式
  - 可编程的软启动序列和关机序列
  - 可编程稳压器配置
  - 24通道模拟多路复用器：用于智能系统监控/诊断
- 一次性可编程（OTP）存储器用于设备配置
- 监测电路符合ASIL B安全等级
  - 独立电压监测：可编程故障保护
  - 高级热监测和保护
  - 外部看门狗监测：可编程内部看门狗计数器
  - I2C CRC和写保护机制
  - 模拟内置自检（ABIST）

1.4. 电源域描述

单板共分成两个电源域，PMIC FS85为M域系统电源、M域外设、PMIC PF82电源供电，PMIC PF82为A域系统电源、A域外设电源供电，具体电源树如下图所示。

版权所有 © 上海云集互联有限公司

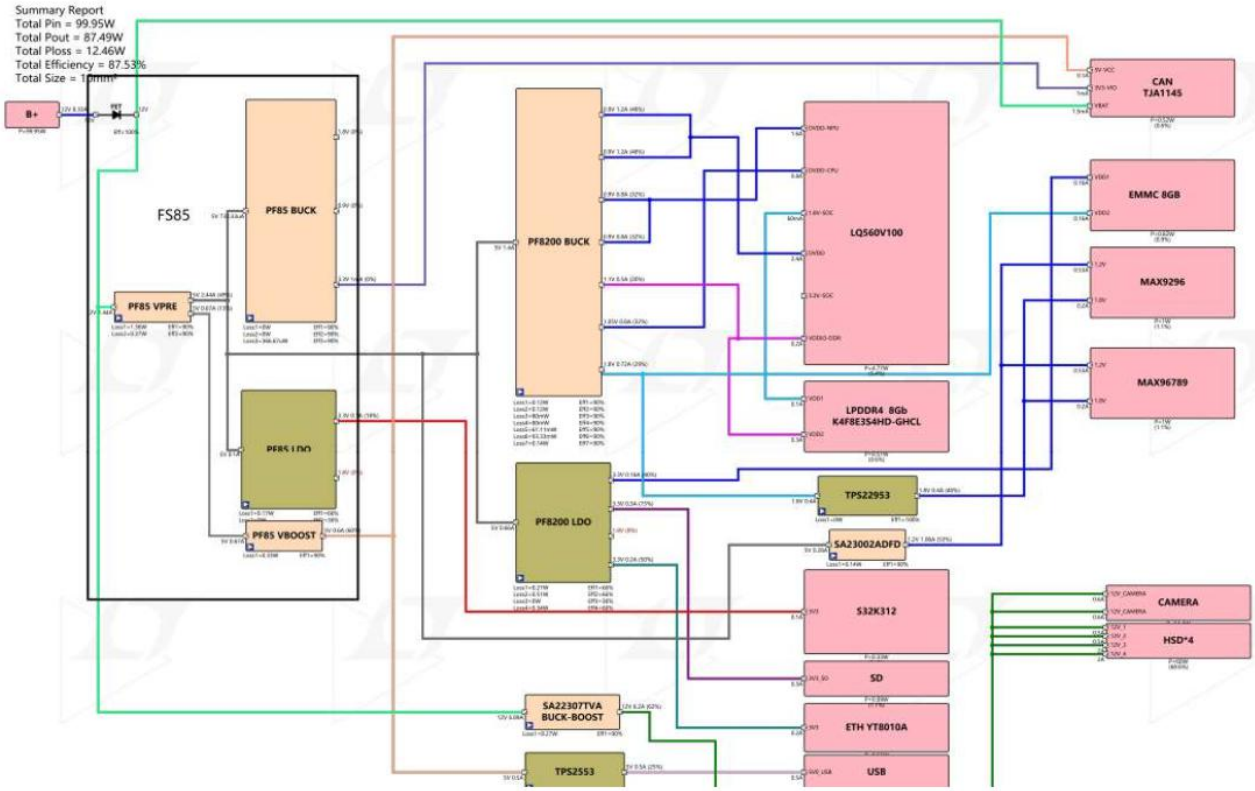


图 1- 3 PMIC电源树

版权所有 © 上海云集互联有限公司

2. 配置说明

2.1. FS85配置

FS85有两个独立的逻辑单元，main状态机 和 fail-safe状态机。main状态机管理电源管理、待机模式和唤醒源。Fail-safe状态机管理电源管理的监控、单片机的监控和外部IC的监控。FS85 的启动分成 2 个同步的流程：Main 和 Fail-safe。在上电的过程中，FS85 在VSUP电压高于6.5V之后，首先会检查DBG管脚和WAKE1/2管脚的电压。

1) 当 WAKE1 和 WAKE2 管脚都为 0V 时，FS85 仍然处在 Power Down 状态。
2) 当 WAKE1 和 WAKE2 其中一个管脚电压升到 4V 之后，VBOS 开始上电。FS85 进入 STANDBY 状态。如果 DBG 管脚电压为 5V 时，FS85 进入 Debug 模式。
3) main 启动流程中：FS85 进入 STANDBY 状态之后，首先等待 LBIST 自检完成，然后，开始按照顺序输出 VPRE、VBOOST、以及其他电源电压。
4) 在 Fail-safe 启动流程中：FS85 进入 STANDBY 状态之后，FS85 首先执行逻辑量内部自检（LBIST），LBIST 在 6ms 内检查完成。然后，开始执行模拟量内部自检 1（ABIST1）。ABIST1 的自检内容由 OTP 决定。在执行 ABIST1 完成之前，RSTB、FS0B、PGOOD 三个管脚都是 0V。如果 ABIST1 发现错误就会回到 Power Down 模式。如果 ABIST1 执行完成并无错误之后，进入 INIT_FS 状态，并且 PGOOD 和 RSTB 被释放，此时，MCU 开始运行。
5) 对于非 Debug 模式，FS85 进入 INIT_FS 状态之后，会给一个 256ms 的窗口期。而 Debug 模式则没有这个窗口期。此时，PGOOD 和 RSTB 被释放，MCU 可以对 FS85 的寄存器进行操作，比如模拟量内部自检 2（ABIST2）的参数设置、看门狗时间窗口设置等。
6) 当 MCU 在 256ms 内喂一次看门狗，FS85 即可退出 INIT_FS 模式，ABIST2 开始自检，进入 Normal 模式，并且 MCU 需要定时喂狗。
7) 如果 MCU 没有在 256ms 内喂狗，FS85 将会拉低 RSTB，10ms 之后再释放 RSTB，并持续 256ms。如果在连续释放 4 次 RSTB 之后，也就是 1s 之后，FS85 都没有被喂狗，那么，FS85 将会回到 Power Down 状态。

根据上述描述可知，FS85配置主要涉及上电256ms内喂狗和进入Normal模式后定时喂狗两个方面。

2.2. PF82配置

PMIC PF82为A域系统电源、A域外设电源供电，PF82配置主要涉及DDR Traning之前调压及linux驱动开发。DDR Traning之前调压主要调节PF82的sw1、sw2、sw3、sw4和sw6五路电压，此配

版权所有 © 上海云集互联有限公司

置在otfl中进行，具体配置可以参考lq560v100_sdk/main/src/boot/otfl/drivers/pf8x目录下源码。

2.2.1. PF82 linux驱动介绍

PMIC PF82 linux驱动用到的内核模块主要有MFD框架、Regmap框架、Regulator框架。

2.2.1.1. MFD框架

MFD（Multi-function Device）多功能设备，许多有共性的设备的集合，MFD子系统是Linux下一种用于管理和控制多功能设备的软件框架。他提供一种统一接口，使得多个设备可以通过一个驱动程序进行管理和控制。MFD是多个具有类似功能设备的集合，每一个功能称为一个cell。MFD为每个cell创建一个platform设备，因为某些设备的共性，所以可以在MFD中提供共同的函数给其下子设备进行调用。

整个MFD包括3部分：

- MFD Core提供MFD设备的注册和去注册，以及对Cell的操作。
- MFD设备驱动。
- MFD Cell描述符，一个描述符对应一个MFD设备单元。MFD为每个Cell创建一个Platform设备。

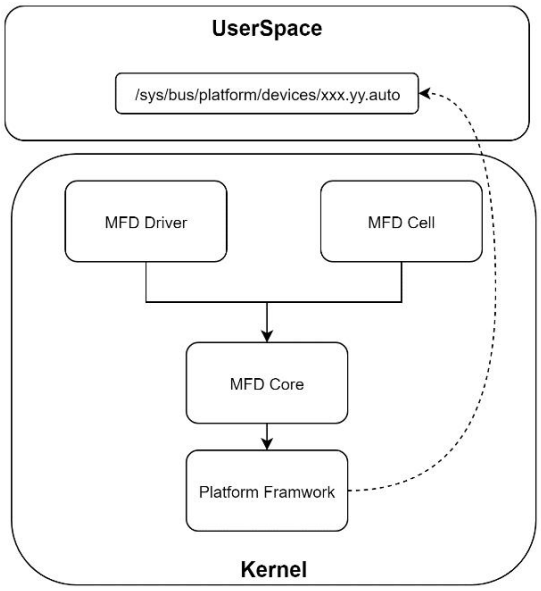


图 2- 1 MFD框架

MFD常用于PMIC设备，主要功能包括：

- MFD对所属设备进行统一管理，包括使能和去使能，以及suspend/resume。
- MFD进行统一中断处理。
- MFD负责设备注册和注销。

版权所有 © 上海云集互联有限公司

PF82的MFD驱动框架代码可以参考lq560v100_sdk/main/src/platform/linux/linux-5.10.y/drivers/mfd/pf8x00-core-oritek.c源文件。

2.2.1.2. Regmap框架

基于代码复用的原则，linux在3.1内核后引入了regmap框架，将寄存器访问的共同逻辑抽象出来，驱动开发人员不需再去纠结使用 SPI 或者 I2C 接口 API 函数，统一使用 regmap API 函数。这样的好处就是统一使用 regmap，降低了代码冗余，提高了驱动的可以移植性。regmap 的重点在于，通过 regmap 框架提供的统一接口函数来访问器件的寄存器，SOC 内部的寄存器也可以使用 regmap 接口函数来访问。

regmap是在linux内核为减少慢速I/O驱动上的重复逻辑，提供一种通用的接口来操作底层硬件寄存器的模型框架。此外，regmap在驱动和硬件寄存器之间增加了cache，减少底层低速I/O的操作次数，提高访问效率；当然实时性会有所降低。regmap 驱动框架如下图所示：

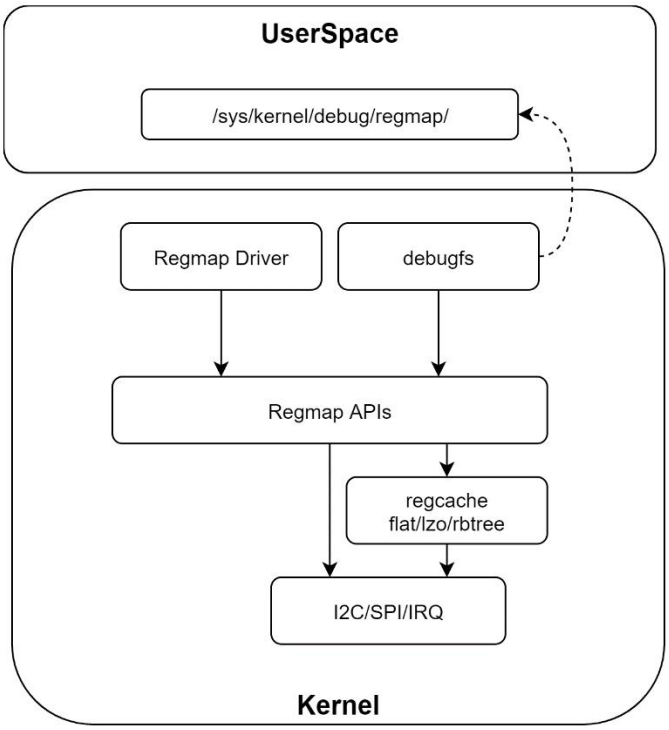


图 2- 2 regmap框架

regmap 框架分为三层：

- 底层物理总线：regmap 就是对不同的物理总线进行封装，目前 regmap 支持的物理总线有 i2c、i3c、spi、mmio、sccb、sdw、slimbus、irq、spmi 和 w1。
- regmap 核心层：用于实现 regmap，我们不用关心具体实现。
- regmap API 抽象层：regmap向驱动编写人员提供的API接口，驱动编写人员使用这些API接口来操作具体的芯片设备，也是驱动编写人员重点要掌握的。

版权所有 © 上海云集互联有限公司

对于使用regmap框架来说，可以不用关心regmap core的实现过程，只需根据物理总线类型，配置好相关参数信息，即可调用regmap api访问芯片寄存器。

PF82的Regmap驱动框架代码可以参考lq560v100_sdk/main/src/platform/linux/linux-5.10.y/drivers/mfd/pf8x00-i2c-oritek.c源文件。

2.2.1.3. Regulator框架

regulator framework的目的很直接：提供标准的内核接口，控制系统的voltage/current regulators，并提供相应的机制，在系统运行的过程中，动态改变regulators的输出，以达到省电的目的。

regulator framework的软件架构如下：

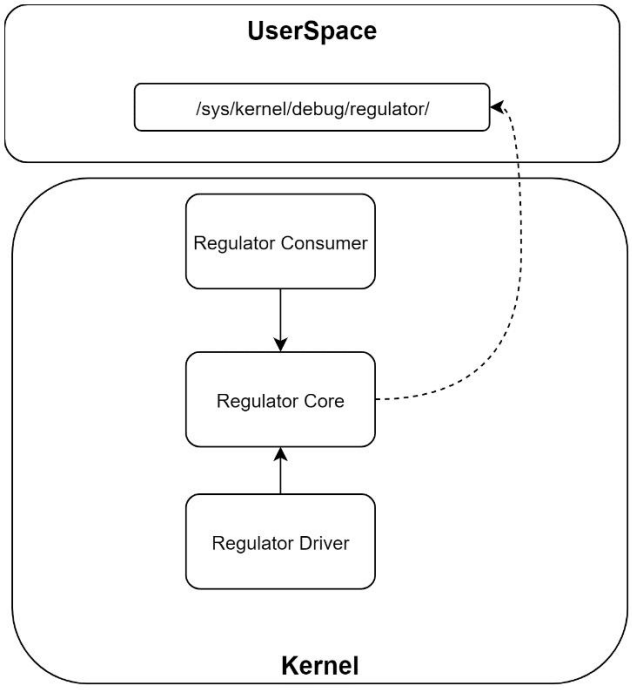


图 2- 3 regulator框架

1) machine

machine使用struct regulator_init_data，静态的描述regulator在板级的硬件连接情况，这些限制通过驱动或dts配置，涉及到系统供电安全，因此必须小心。

2) driver

driver模块的功能，是从regulator driver的角度，抽象regulator设备。

- 使用struct regulator_desc，描述regulator的静态信息，包括：名字、supply regulator的名字、中断号、操作函数集（struct regulator_ops）、使用regmap时相应的寄存器即bitmap等。
- 使用struct regulator_config，描述regulator的动态信息（所谓的动态信息，体现在struct regulator_config变量都是局部变量，因此不会永久保存），包括struct regulator_init_data指针、设备指针、enable gpio等。

版权所有 © 上海云集互联有限公司

- 提供regulator的注册接口（regulator_register/devm_regulator_register），该接口接受描述该regulator的两个变量的指针：struct regulator_desc和struct regulator_config，并分配一个新的数据结构（struct regulator_dev，从设备的角度描述regulator），并把静态指针（struct regulator_desc）和动态指针（struct regulator_config）提供的信息保存在其中。
- regulator driver以struct regulator_dev（代表设备）指针为对象，对regulator进行后续的操作。

3) consumer

consumer的功能，是从regulator consumer的角度，抽象regulator设备（struct regulator），并提供regulator操作相关的接口。

4) core

core负责上述逻辑的具体实现，并以sysfs的形式，向用户空间提供接口。

PF82的Regulator驱动框架代码可以参考lq560v100_sdk/main/src/platform/linux/linux-5.10.y/drivers/regulator/pf8x00-regulator-oritek.c源文件。

2.2.2. PF82 linux总体框架

2.2.2.1. 软件操作

PMIC PF82稳压器客户端驱动通过重新配置 PMIC 硬件控制寄存器来执行操作，某些PMIC电源管理操作取决于系统设计和配置。例如，如果系统由 PMIC 以外的电源供电，那么关闭或调整 PMIC 稳压器将不起作用。相反，如果系统由PMIC供电，那么使用电源管理驱动和稳压器客户端驱动的任何更改都会影响整个系统的运行或稳定性。

2.2.2.2.驱动特性

PMIC PF82驱动基于稳压器核心驱动。它为 PMIC 组件的稳压器控制提供以下服务：

- 打开/关闭所有稳压器。
- 设置所有稳压器的值。
- 获取所有稳压器的当前值。

2.2.2.3.稳压器 API

稳压器电源架构旨在为 Linux 内核中的稳压器和稳流器提供通用接口。它旨在为客户端或消费者驱动提供电压和电流控制，并通过 sysfs 接口向用户空间应用提供状态信息。其目的是允许系统动态控制稳压器输出，以节省电力并延长电池寿命。这适用于稳压器（电压输出可控）和电流吸收器（电流输出可控）。

在此框架下，大多数电源操作可以通过以下统一的 API 调用完成：

版权所有 © 上海云集互联有限公司

- regulator_get是一个统一API调用，用于查找和获取稳压器的参考
- regulator_put是一个统一API调用，用于释放稳压器源
- regulator_enable是启用稳压器输出的统一API调用
- regulator_disable是用于禁用稳压器输出的统一API调用
- regulator_is_enabled是指启用的稳压器输出
- regulator_set_voltage是用于设置稳压器输出电压的统一API调用
- regulator_get_voltage是用于获取稳压器输出电压的统一API调用

可在 Linux 内核中的regulator核心源代码中找到更多 API 和详细信息，位于：

sdk/build/main/richos/linux/platform/linux/linux-5.10.116/drivers/ regulator/core.c

2.2.2.4.驱动架构

下图显示了PMIC PF82驱动的基本架构。

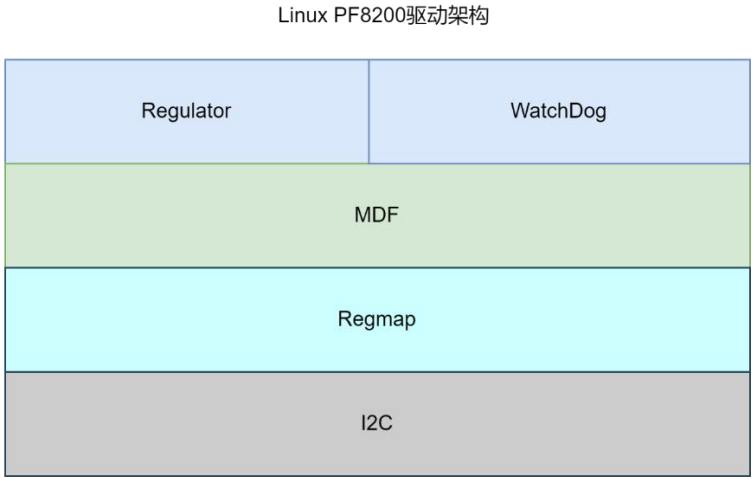


图 2- 4 linux pf8200驱动架构

2.2.3. PF82 linux驱动配置

2.2.3.1.dts配置

PMIC PF82 包含 regulator，watchdog。 PF82 的配置是通过设备树进行配置。PF82在内核中的设备是多个设备（MFD）同时存在，并存在层次对应关系。PF82的设备树配置如下所示。

```
&i2c_bus2 {
    status = "okay";
    #address-cells = <1>;
    #size-cells = <0>;

    pmic@8 {
```


版权所有 © 上海云集互联有限公司

```
        compatible = "nxp,pf8200";
        reg = <0x8>;

        regulators {
            reg_ldo1: ldo1 {
                regulator-min-microvolt = <1500000>;
                regulator-max-microvolt = <5000000>;
                regulator-boot-on;
                regulator-always-on;
            };

            reg_ldo2: ldo2 {
                regulator-min-microvolt = <1500000>;
                regulator-max-microvolt = <5000000>;
                regulator-boot-on;
                regulator-always-on;
            };

            reg_ldo3: ldo3 {
                regulator-min-microvolt = <1500000>;
                regulator-max-microvolt = <5000000>;
                regulator-boot-on;
                regulator-always-on;
            };

            reg_ldo4: ldo4 {
                regulator-min-microvolt = <1500000>;
                regulator-max-microvolt = <5000000>;
                regulator-boot-on;
                regulator-always-on;
            };

            reg_buck1: buck1 {
                regulator-min-microvolt = <400000>;
                regulator-max-microvolt = <1800000>;
                regulator-boot-on;
                regulator-always-on;
            };

            reg_buck2: buck2 {
                regulator-min-microvolt = <400000>;
                regulator-max-microvolt = <1800000>;
                regulator-boot-on;
                regulator-always-on;
            };
```


版权所有 © 上海云集互联有限公司

```
        reg_buck3: buck3 {
            regulator-min-microvolt = <400000>;
            regulator-max-microvolt = <1800000>;
            regulator-boot-on;
            regulator-always-on;
        };

        reg_buck4: buck4 {
            regulator-min-microvolt = <400000>;
            regulator-max-microvolt = <1800000>;
            regulator-boot-on;
            regulator-always-on;
        };

        reg_buck5: buck5 {
            regulator-min-microvolt = <400000>;
            regulator-max-microvolt = <1800000>;
            regulator-boot-on;
            regulator-always-on;
        };

        reg_buck6: buck6 {
            regulator-min-microvolt = <400000>;
            regulator-max-microvolt = <1800000>;
            regulator-boot-on;
            regulator-always-on;
        };

        reg_buck7: buck7 {
            regulator-min-microvolt = <1800000>;
            regulator-max-microvolt = <1800000>;
            regulator-boot-on;
            regulator-always-on;
        };

        reg_vsnvs: vsnvs {
            regulator-min-microvolt = <1800000>;
            regulator-max-microvolt = <3300000>;
            regulator-boot-on;
        };
    };

    watchdog {
        compatible = "nxp,pf8x00-watchdog";
        timeout-sec = <32>;
    };
```


版权所有 © 上海云集互联有限公司

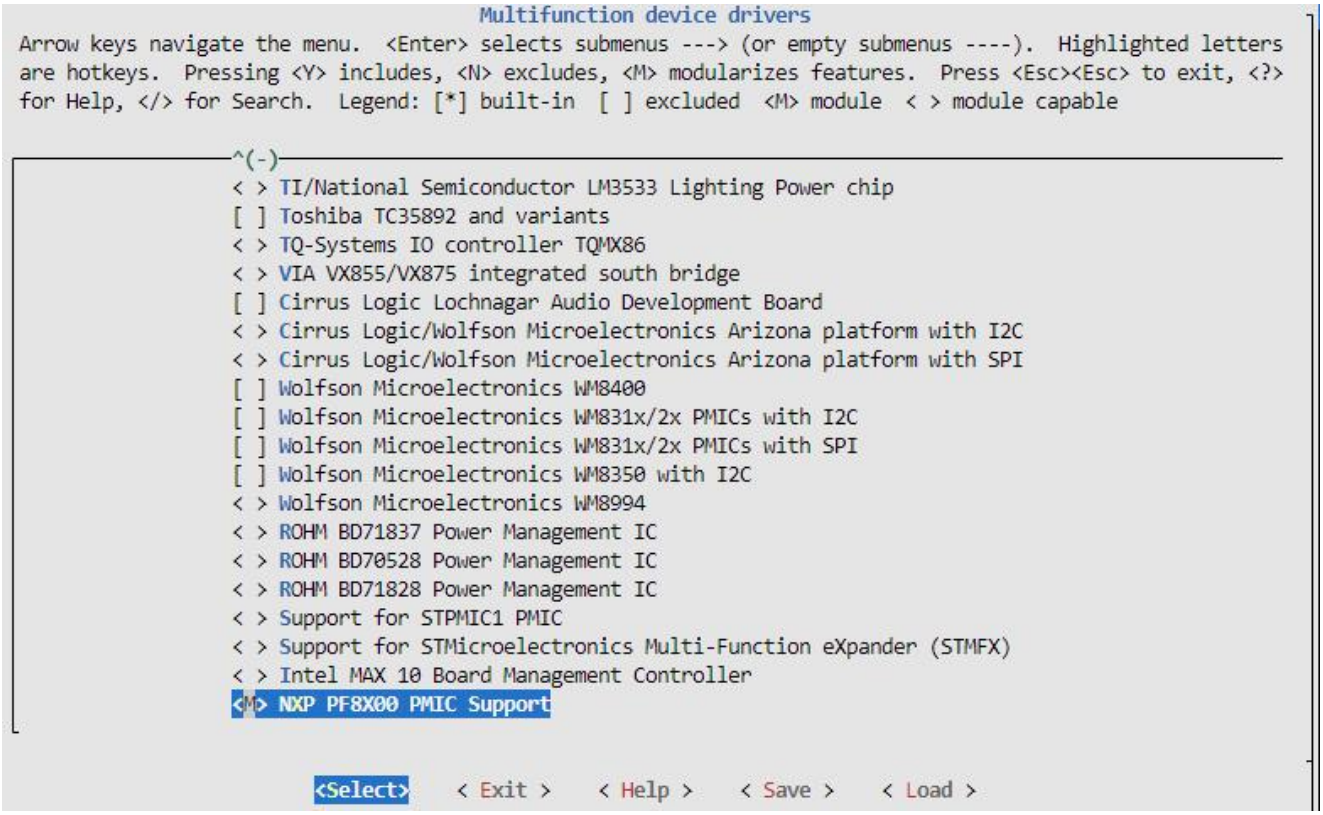
```
    };
};
```

2.2.3.2. kernel menuconfig配置

在main/src/platform/linux下执行make BOOT_MEDIA=spi/emmc menuconfig。SD3589CV100的pinmux是在linux内核启动之后，以KO的方式进行加载的。PMIC驱动在注册过程中会读Device ID，因此不能把PMIC驱动程序直接编译进内核，需要动态加载，以KO方式加载。进入配置主界面，并按以下步骤操作：

1) PMIC

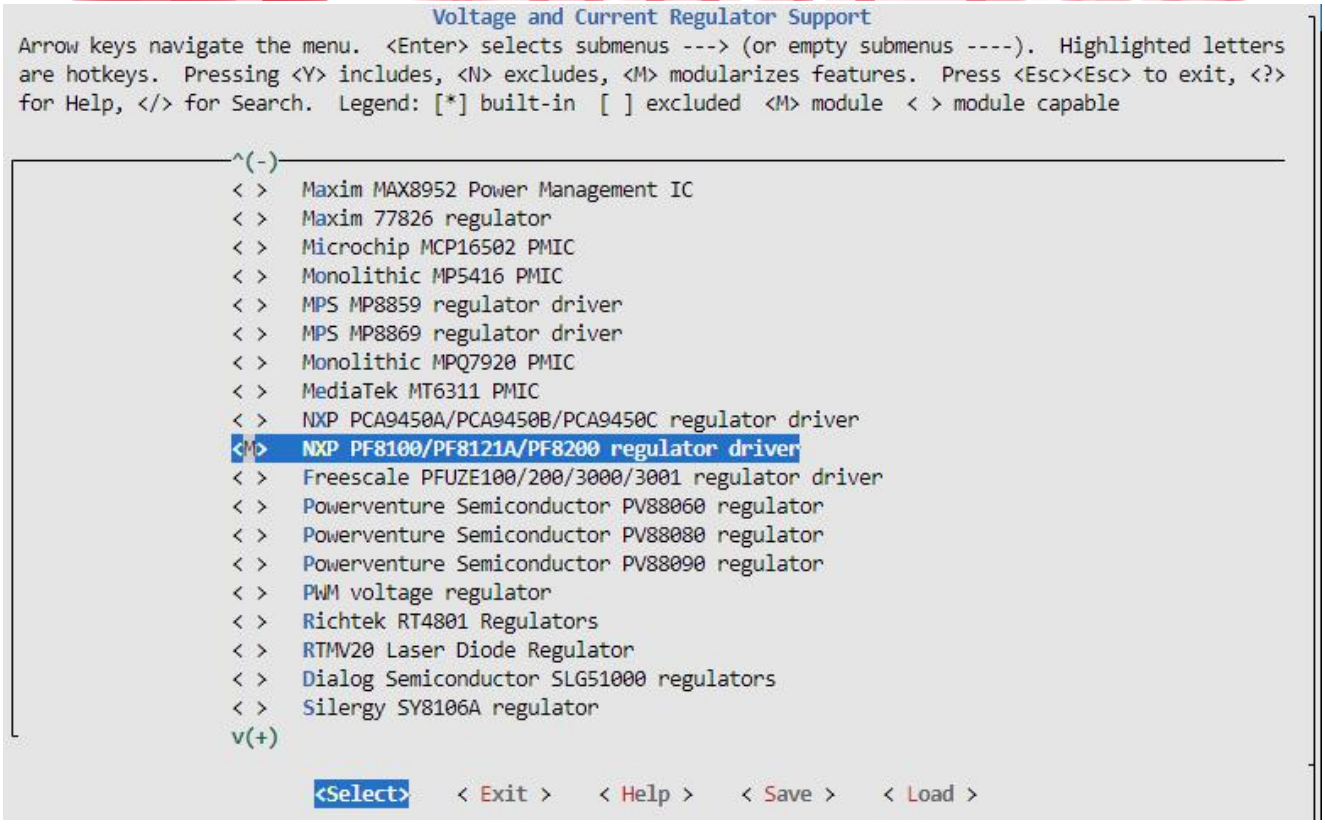
```
Location:
  -> Device Drivers
    -> Multifunction device drivers
      -> <*> NXP PF8X00 PMIC Support
```



2) regulator

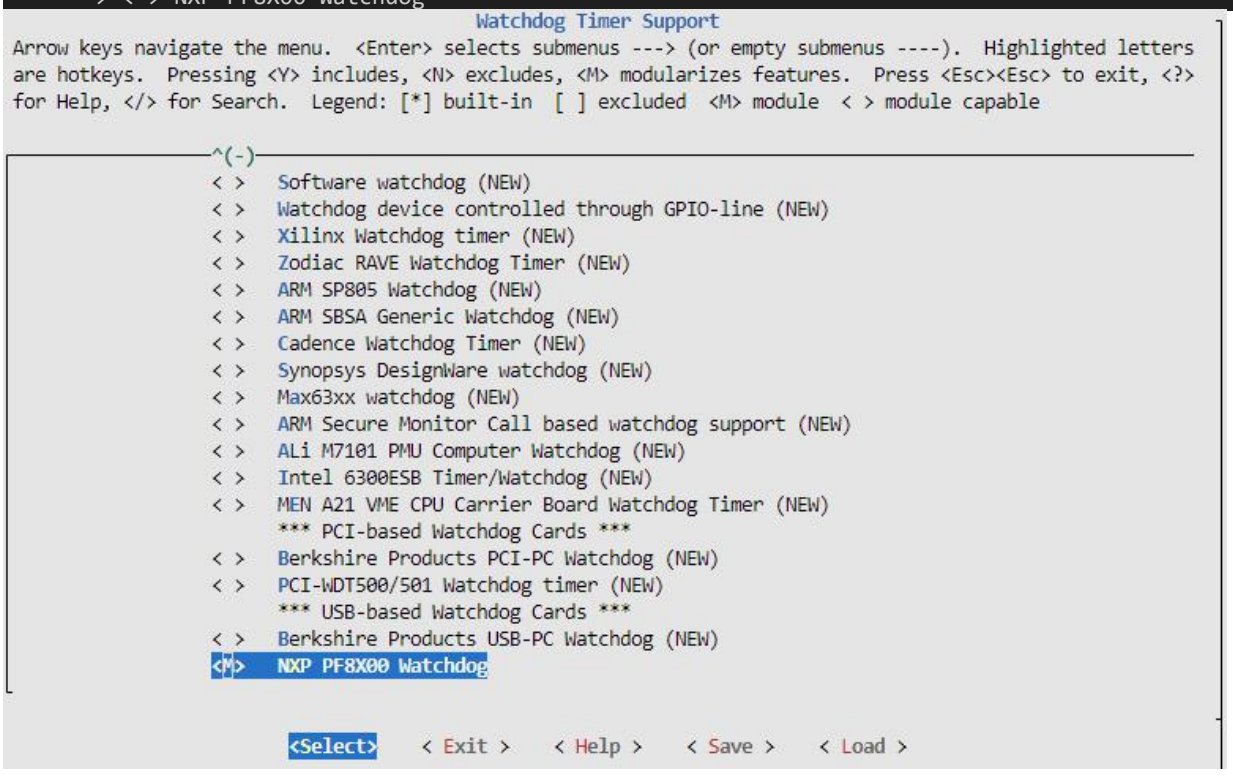
```
Location:
  -> Device Drivers
    -> Voltage and Current Regulator Support
      -> <*> NXP PF8100/PF8121A/PF8200 regulator driver
```


版权所有 © 上海云集互联有限公司



3) watchdog

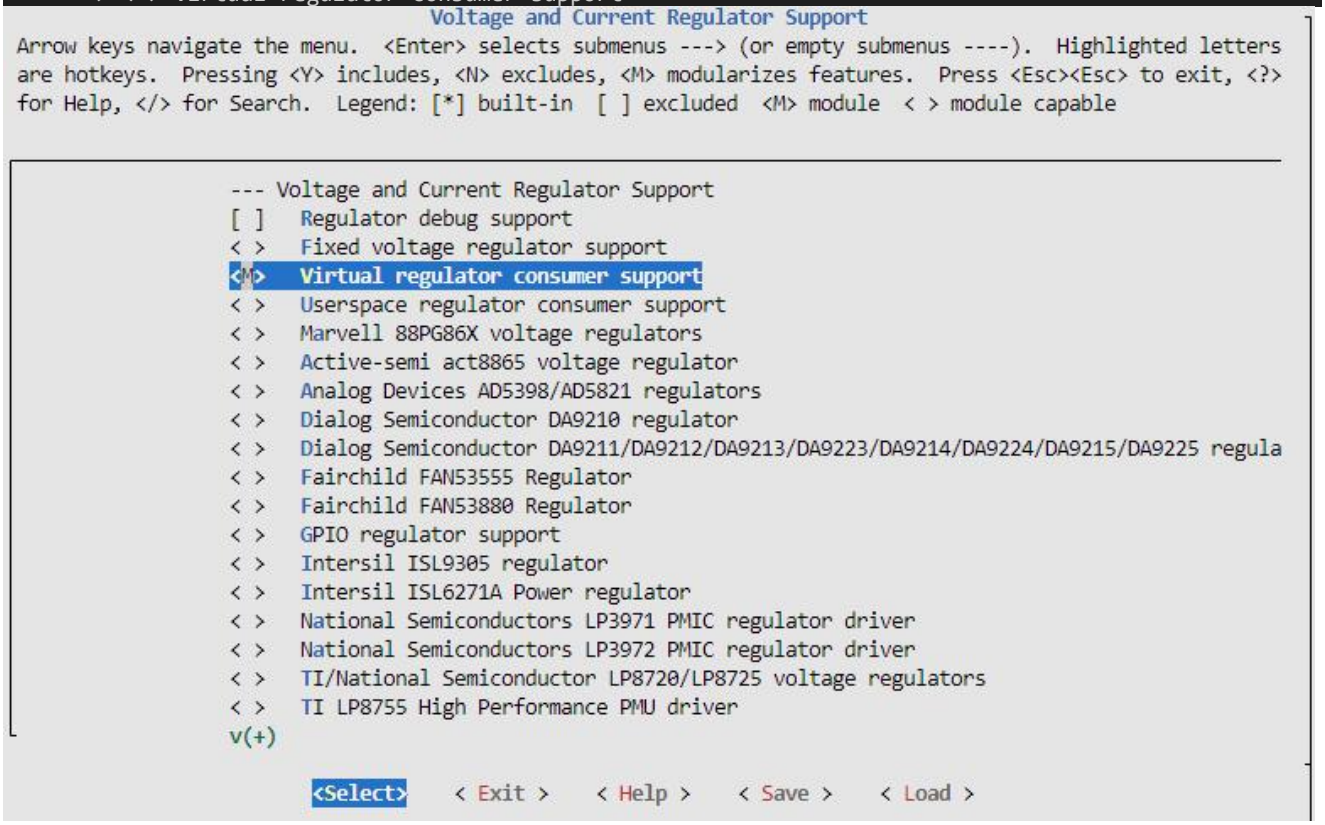
```
Location:
  -> Device Drivers
    -> Watchdog Timer Support
      -> <*> NXP PF8X00 Watchdog
```



版权所有 © 上海云集互联有限公司

4) virtual regulator

```
Location:
  -> Device Drivers
    -> Voltage and Current Regulator Support
      -> <*> Virtual regulator consumer support
```



2.2.3.3.使用配置

1) regulator 使用 demo

其他设备对regulator_dev设备的引用通过设备树配置

```
    <name>-supply = <&reg_ldo1>;
```

设备中通过对name的获取，可以获取reg_dcdc1的regulator_dev设备，然后对此路电源进行电源开关，电压设置等功能。具体设备引用参考内核的regulator使用文档。

2) watchdog 使用 demo

PMIC 会创建一个看门狗，创建的 watchdog 会在/dev/watchdog*，使用方法如下。

- pmic 的 watchdog 打开后即开启，关闭文件不能关闭看门狗。
- 看门狗使用标准的 set_timeout 方法设置看门狗时间。
- 通过写 watchdog 可以喂狗，或者使用标准的 ioctl 方法。

2.3. 编译配置

版权所有 © 上海云集互联有限公司

为支持PMIC调压，需要修改otfl相应配置，具体对应代码：

文件路径：lq560v100_sdk/main/src/boot/otfl/cfg.mk，将#CFG_PMIC_SET = 1/* PMIC调压 */一行放开注释。

2.4. 烧录配置

FS85进入DEBUG模式后看门狗功能被禁用，在该模式下才可以进行镜像烧录，具体操作如下：

1) 点击烧写工具烧写按钮

2) 按住FS85-DBG按键

3) 上电并松开FS85-DBG按键

4) 等待烧写完成

版权所有 © 上海云集互联有限公司

3. 调试说明

3.1. FS85调试说明

PMIC FS85为M域系统电源，主要由外接MCU控制，在RTOS下可以通过调用NXP提供的电压接口进行电压状态控制。

3.2. PF82调试说明

3.2.1. OTFL PF82调试说明

在OTSL中会调节PF82的sw1、sw2、sw3、sw4和sw6五路电压，调节后可以通过示波器或者万用表量取电压值是否合理，具体可以参考lq560v100_sdk/main/src/boot/otfl/drivers/pf8x目录下源码。

3.2.2. Linux PF82调试说明

3.2.2.1. regmap registers 节点

shell 命令读写寄存器。寄存器调试是指直接对 PMIC PF82的寄存器进行读写操作，此操作应该对寄存器有了解的情况下进行操作，不正确的操作方式将会导致芯片烧毁。在终端中，对抛出的调试节点进行读写操作，即可对寄存器进行读写操作。无论是读还是写寄存器，都应该首先挂载 debugfs 文件系统。由于 PMIC PF82是通过 regmap 进行读写操作，应该可以使用 regmap 的调试节点进行对 PMIC 的读写访问操作。 regmap 的调试节点在 debugfs 文件系统下面，通过对 regmap 调试节点的操作可以对 PMIC 的寄存器进行读写访问操作。

drivers/base/regmap/regmap-debugfs.c文件提供在debug_fs下写操作，需要在源码里修改，该操作是危险的，对于像PMIC这样的设备。

寄存器调试挂载在debugfs文件系统。

mount -t debugfs none /sys/kernel/debug

- 写操作

echo ${reg} ${value} > /sys/kernel/debug/regmap/${dev-name}/registers

实例：

echo 0x43 0x0f > /sys/kernel/debug/regmap/2-0008/registers (写0x43寄存器值为0x0f)

- 读操作

cat /sys/kernel/debug/regmap/${dev-name}/registers

实例：

版权所有 © 上海云集互联有限公司

cat /sys/kernel/debug/regmap/2-0008/registers (读取pmic所有寄存器)

3.2.2.2. max_microvolts 和 min_microvolts 节点

shell 命令设置 regulator 电压。对各路电压的控制是常用的调试手段，通过对各路不同电压设置，实现功耗，性能，稳定性等信息。内核通过对每一路电源创建一个 virtual 设备，通过导出 virtual 设备的电源控制节点，用来对每一路电源的电压进行控制。通过进入不同的 virtual 设备，来控制不同的电源。virtual 设备存在于 PF82 主设备节点下面，因此设备路径为主设备下面的从设备。

cd /sys/devices/platform/soc/soc:amba/11062000.i2c/i2c-2/2-0008/

```
# ls
driver                pf8x00-watchdog       reg-virt-consumer.3
modalias              power                 regulator
name                  reg-virt-consumer.0   subsystem
of_node               reg-virt-consumer.1   uevent
pf8x00-regulators     reg-virt-consumer.2
```

reg-virt-consumer.0是sw1，reg-virt-consumer.1是sw2

reg-virt-consumer.2是sw3，reg-virt-consumer.3是sw4

比如，调sw1电压为800mV

cd reg-virt-consumer.0，通过此路径下面的 max_microvolts 和 min_microvolts 设备节点进行写操作，用来完成对设备电源的控制，此例为：

echo 800000 > max_microvolts

echo 800000 > min_microvolts

3.2.2.3. regulator_summary 节点

shell 命令查询 regulator 状态。内核提供调试节点供电源进行调试进行，我们可以通过 kernel 的调试节点获取各路电源的各个详细状态。首先需要 mount debugfs 文件系统。

mount -t debugfs none /sys/kernel/debug

版权所有 © 上海云集互联有限公司

```
# cat /sys/kernel/debug/regulator/regulator_summary
 regulator                       use open bypass  opmode voltage current     min     max
---------------------------------------------------------------------------------------------
 regulator-dummy                   1    0      0 unknown     0mV     0mA     0mV     0mV
 ldo1                              1    0      0 unknown  3300mV     0mA  1500mV  5000mV
 ldo2                              1    0      0 unknown  3300mV     0mA  1500mV  5000mV
 ldo3                              1    0      0 unknown  1800mV     0mA  1500mV  5000mV
 ldo4                              1    0      0 unknown  3300mV     0mA  1500mV  5000mV
 buck1                             1    1      0 unknown   900mV  4500mA   400mV  1800mV
    reg-virt-consumer.0-buck1      0                                0mA     0mV     0mV
 buck2                             1    1      0 unknown   900mV  4500mA   400mV  1800mV
    reg-virt-consumer.1-buck2      0                                0mA     0mV     0mV
 buck3                             1    1      0 unknown   900mV  4500mA   400mV  1800mV
    reg-virt-consumer.2-buck3      0                                0mA     0mV     0mV
 buck4                             1    1      0 unknown   900mV  4500mA   400mV  1800mV
    reg-virt-consumer.3-buck4      0                                0mA     0mV     0mV
 buck5                             1    0      0 unknown  1100mV  4500mA   400mV  1800mV
 buck6                             1    0      0 unknown   600mV  4500mA   400mV  1800mV
 buck7                             1    0      0 unknown  1800mV  4500mA  1800mV  1800mV
 vsnvs                             0    0      0 unknown  3000mV     0mA  1800mV  3300mV
#
```


版权所有 © 上海云集互联有限公司